

Protect Amazon API Applications with Data Encryption

- Home Documentation > API Reference
- Code Samples Announcements Models
- Release Notes FAQ GitHub Videos

- GET STARTED
- Welcome to SP-API Documentation
- What is the Selling Partner API?
- Selling Partner API Onboarding Overview >

- Terminology
- Frequently Asked Questions >

- CHANGELOG
- SP-API Release Notes
- SP-API Deprecation Schedule

Protect Amazon API Applications with Data Encryption

Secure sensitive data in Amazon SP-API Applications.

Customers trust that their data is kept secure, and encryption serves as the mechanism to maintain this confidentiality. Examples of this data include, but are not limited to personally identifiable information (PII), passwords, and proprietary information. Through the use of keys, encryption transforms data from a readable format (plaintext) to an unreadable format (ciphertext). Use the key to encrypt (lock) the data so that only parties with the corresponding key can decrypt (unlock) it. This guide provides guidance to help you secure the sensitive data customers have entrusted to you.

Amazon Data Protection Policy (DPP) requirements

The Amazon Data Protection Policy contains two categories related to security requirements:

- **Section 1. General Security Requirements**, which includes requirements for all solution providers building SP-API applications.
- **Section 2. Additional Security Requirements for Personally Identifiable Information (PII)**, which includes additional requirements for solutions providers performing restricted operations on SP-API that involve personally identifiable information (PII).

DIRECTORIES

SP-API Models

SP-API Seller Use Cases

SP-API Vendor Use Cases

Seller Central URLs

Vendor Central URLs

REGISTRATION

SP-API Registration Overview >

Developer Registration Request Status

Third-Party Provider Registration >

Service Provider Registration and Role Approval Process Overview

Website Guidelines for Public Developers and Service Providers

User Permissions for Service Providers

Register your Application

Update your Application Information

Rotate your Application's LWA Credentials >

View your Application Information and Credentials

Learn How Sellers Authorize Service Providers

Selling Partner API Roles >

Policies and Agreements

About Facial Data

AUTHORIZATION

Authorize Applications >

Renew Authorizations

Revoke Authorizations

Authorization Limits

To comply with the **data encryption** requirements in the DPP, Solution providers must:

- Create data classification documents that identify all data that requires encryption.
- Use AES-128 bit or larger keys (AES-256 preferred) or RSA-2048 bit keys (or higher) to encrypt all PII data.
- Apply encryption to end-device drives, servers, databases, and backup storage systems.
- Encrypt SP-API keys and credentials and never expose them in plain text.
- Deploy Key Management Systems (KMS) that cover the complete key lifecycle, including key generation, exchange, secure storage, and processes for key revocation and rotation.
- Use cryptographically secure methods to generate keys and store the keys in dedicated KMS with access controls.
- Rotate encryption keys at least annually and revoke compromised keys immediately.

Encryption in transit: Solution providers must encrypt all Amazon Information in transit (for example, when the data traverses a network, or is otherwise sent between hosts). Use HTTP over TLS 1.3 (HTTPS), which offers security improvements and performance advantages over earlier versions. Solution providers must enforce this security control on all applicable external endpoints that customers use as well as internal communication channels (for example, data propagation channels among storage layer nodes, connections to external dependencies) and operational tooling. Solution providers must disable communication channels which do not provide encryption in transit even if unused (for example, removing the related dead code, configuring dependencies only with encrypted channels, and restricting access credentials to use of encrypted channels).

Solution providers must use data message-level encryption (such as using AWS

Special Authorization Types

>

Protect Amazon API Applications with Data Encryption

Encryption SDK) where channel encryption (such as TLS) terminates at untrusted multi-tenant hardware (such as untrusted proxies).

INTEGRATION

Building Robust Amazon SP-API Applications

SP-API SDKs

>

Encryption at rest: Solution providers must encrypt all PII at rest (for example, when you persist data) using industry best practice standards (such as using AES-128 bit or larger keys, AES-256 bit is preferred, or RSA with 2048-bit key size or higher). The cryptographic materials (such as encryption/decryption keys) and cryptographic capabilities (such as daemons implementing virtual Trusted Platform Modules and providing encryption/decryption APIs) used for encryption of PII at rest must be only accessible to the Solution provider's processes and services. Solution providers must not store PII in removable media (such as USB) or unsecured public cloud applications (for example, public links made available through Google Drive).

Solution providers must securely dispose of any printed documents containing PII.

Foundation of data encryption

Protecting critical information is essential for Solution providers operating at scale. Securing data through encryption transforms the content in a manner that makes it unreadable without a secret key to decrypt the content back to readable format. Encryption encodes plaintext into ciphertext using a cryptographic key. To encrypt a message (data), a key is needed. Data encryption uses a key (also called a secret) to encrypt and decrypt messages. Only the person/device with the correct key can decrypt the message and read its contents. Encryption helps keep Amazon Information safe. Even if a malicious user obtains the ciphertext, they won't be able to read the contents of the message without having the key to decrypt it. In addition, these cryptographic keys must be protected throughout their lifecycle to prevent unauthorized use, access, disclosure, and modification. Keep in mind that there is a difference between hash functions and encryption. A hash function is a cryptographic algorithm that

Delete an Application From Your Developer Account

CALLING THE SP-API

Configuration Details

>

Call Structure

>

Development Tools

>

Performance Management

>

SOLUTION PROVIDER PORTAL

Manage Users in Solution Provider Portal

Manage Your Business And Contact information in Solution Provider Portal

Update Your Developer Profile in Solution Provider Portal

Unify Your Developer Accounts

Verify Your Identity in Solution Provider Portal

API Usage Metrics in Solution Provider Portal

TUTORIALS

Tutorial: Subscribe to the ORDER_CHANGE Notification

Tutorial: Test Selling Partner API Endpoints

Tutorial: Automate your SP-API Calls Using a C# SDK

Tutorial: Automate your SP-API Calls Using a Java SDK

Tutorial: Automate your SP-API Calls Using a JavaScript SDK for Node.js

Tutorial: Automate your SP-API Calls Using a Python SDK

Tutorial: Automate your SP-API Calls using a prebuilt C# SDK

Tutorial: Automate your SP-API Calls using prebuilt Java SDK

Tutorial: Automate your SP-API Calls using a prebuilt JavaScript SDK

Tutorial: Automate your SP-API Calls using prebuilt PHP SDK

Tutorial: Automate your SP-API calls using a prebuilt Python SDK

Tutorial: Authorize Multiple Vendor Central Accounts with a Single SP-API Application

Tutorial: Retrieve Merchant Shipping Templates

Tutorial: Grant the SP-API Permission to an Amazon SQS Queue

Tutorial: Retrieve and Pass a Purchase Order Number to a Carrier

TROUBLESHOOTING

Troubleshoot SP-API Errors

URL Encoding

Resolve Common HTTP and Authorization Error Codes

External Fulfillment Errors 

Listings Items API Issues Troubleshooting 

SELLING PARTNER APPSTORE

What is the Selling Partner Appstore?

List Your App on the Selling Partner Appstore

Protect Amazon API Applications with Data Encryption

transforms data of arbitrary size into a fixed-size hash value. Hashing verifies data integrity. Encryption, on the other hand, is a two-step process that encrypts and decrypts a message. Encryption protects data by rendering it unintelligible to unauthorized access. Encrypt data both at rest and in transit. Solution providers who use NIST 800-53 as their framework can refer to System and Communications Protection (SC) for specific guidance on protecting their environments. One of the primary NIST controls is [SC-13 Cryptographic Protection](#). SC-13 identifies specific controls, providing additional details to help Solution providers understand SP-API encryption requirements.

Solution providers must use encryption to protect the Amazon Information and customer PII. The following sections describe the critical components of data encryption and provide recommendations to help Solution providers comply with the SP-API Data Protection Policy (DPP) encryption requirements.

Data classification

Data classification provides a way to categorize organizational data based on levels of sensitivity. It includes understanding what data types are available, where the data is stored, access levels, and the security of data. Amazon information falls into two categories: PII and non-PII. PII is any information that identifies an Amazon customer. Non-PII data is any other Amazon data, for example product listing information. Solution providers can protect Amazon Information by carefully managing an appropriate data classification system, level of access, and encryption of data meeting the DPP requirements. Encrypt Amazon information that identifies customers, store it securely, and restrict access to authorized users only. Amazon recommends a defense-in-depth approach and reduced human access to Amazon customer PII. Solution providers must have strong authentication mechanisms in their applications. In addition, Solution providers must ensure that all connections to their applications

[Edit Your Appstore Listing](#)[Check Listing Status](#)[Appstore Ratings and Reviews](#)[Press Releases and Promotions](#)**SERVICE PROVIDER NETWORK**[List Your Service](#)**SECURITY AND COMPLIANCE**[SP-API Security and Compliance Overview](#)[Amazon Selling Partner API Guard Implementation Guide](#)

>

[VAT Calculation Service](#)

>

[Amazon Seller Data Access](#)[Best Practices](#)

>

[Key Security Control Guidance](#)[Logging and Monitoring for Amazon SP-API Applications](#)[Network Protection Guidance](#)[Protect Amazon API Applications with Data Encryption](#)[Protect Amazon API Applications with Data Retention and Recovery](#)[Protect Amazon SP-API Applications with Incident Response](#)[Safeguarding Sensitive Credentials](#)

Protect Amazon API Applications with Data Encryption

originate from a trusted network and have the required access.

Data integrity

It is important for Solution providers to ensure that the Amazon Information in their systems maintains its integrity as it moves through their systems. Cryptographic hash functions are a reliable way to establish data integrity. A hash function is an irreversible function that maps original data of arbitrary size to a fixed-size hash value. If a Solution provider offers Amazon Information for download, there are two main threats to its integrity: accidental data changes from network or storage issues and intentional tampering by attackers. Solution providers can mitigate these threats by using hashing functions to verify file integrity. For example, a Solution provider can hash the contents of a file before making it available for download. When the file is uploaded back into the application, the Solution provider can run the hash function on the uploaded file and compare the hash output with that of the original file to verify the file's integrity. This provides the best protection against data changes, whether they are accidentally caused by network or storage issues or intentionally caused by an attacker. Solution providers can increase their confidence in data integrity by verifying a hash with multiple hash functions.

Encryption in transit

Data transmitted between systems is susceptible to undesired interference by unauthorized users or third-parties. Data transmission can be limited to a Solution provider's private or public networks and still be susceptible to malicious attacks.

Solution providers must protect Amazon Information in-transit, including in communication between other services and end users, by implementing encryption-in-transit. This helps to protect the confidentiality and integrity of data.

Encryption-in-transit protects information by using supported encryption protocols such as TLS 1.3 (preferred) or TLS 1.2, or HTTPS (HTTP over TLS). TLS 1.3 offers stronger security and improved

Vulnerability Management	
A+ CONTENT API	
A+ Content API	>
A+ Content Examples	
AMAZON WAREHOUSING AND DISTRIBUTION API	
Amazon Warehousing and Distribution API	>
Amazon Warehousing and Distribution API v2024-05-09 Reference	
APP INTEGRATIONS API	
App Integrations API	>
APPLICATION MANAGEMENT API	
Application Management API	>
CATALOG ITEMS API	
Catalog Items API	>
Catalog Items API Rate Limits	
Catalog Items API v2022-04-01 Reference	
CUSTOMER FEEDBACK API	
Customer Feedback API	>
Customer Feedback API Rate Limits	

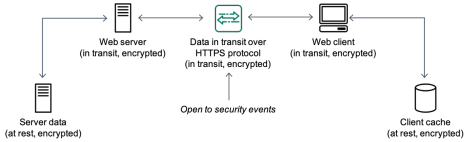
Protect Amazon API Applications with Data Encryption

performance compared to earlier versions. This level of encryption must be enforced across all external and internal endpoints. Data propagating through channels, connections to external dependencies, and operational tooling must be protected with encryption-in-transit. As Amazon Information is transmitted, the communication channel used must be encrypted so that no unauthorized interference is possible. The best practice is to encrypt and authenticate all traffic. In NIST 800-53, encryption-in-transit is outlined in control [SC-8 Transmission Confidentiality and Integrity](#). Solution providers leveraging this control should consider adopting the SC-8 control enhancements, as each enhancement can provide additional protections for data in transit.

Solution providers should consider the following when encrypting data in transit:

- Disable communication channels that cannot support encryption-in-transit.
- Avoid routing traffic through multi-tenant infrastructure nodes.
- Use data message-level encryption if end-to-end TLS is not permissible.

The following figure shows HTTPS protocol in use.



The following table lists some Amazon-compliant security protocols for encrypting data in transit.

Transfer Type	Insecure Protocols (Non-Compliant)	Secure Protocols (Compliant)
Web Access	HTTP	HTTPS with TLS 1.2+
E-Mail Servers	POP3, SMTP, IMAP	POP3S, IMAPS, SMTPS
File Transfer	FTP, RCP	FTPS, SFTP, SCP,

DATA KIOSK

Data Kiosk API

Data Kiosk Schema Explorer User Guide

Data Kiosk Query Processing Finished Notification

Building Data Kiosk workflows guide

Vendor Analytics Dataset Use Case Guide

Schedule Data Kiosk Queries

DELIVERY BY AMAZON API

Delivery by Amazon API

EASY SHIP API

Easy Ship API

EXTERNAL FULFILLMENT

External Fulfillment Inventory API

External Fulfillment Returns API

External Fulfillment Shipping API

External Fulfillment Errors

External Fulfillment APIs Rate Limits

FULFILLMENT BY AMAZON (FBA)

FBA Inventory API

FBA Inventory Dynamic Sandbox Guide

FEEDS API

Transfer Type	Insecure Protocols (Non-Compliant)	Secure Protocols (Compliant)
		WebDAV over HTTPS
Remote Shell	Telnet	SSH-2
Remote Desktop	VNC	r-admin, RDP

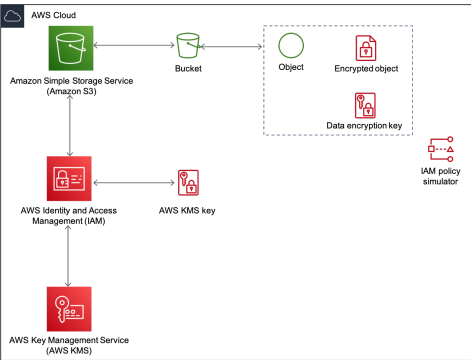
Encryption at rest

Solution providers must protect Amazon information stored in any data store. A data store is any storage medium that retains data for a specified period. Database solutions, block storage, object storage, and archived data blocks are all examples of data stores that must be protected. Solution providers can secure data in a data store using encryption – this is called encryption-at-rest. Implementing encryption and appropriate access controls reduces the risk of unauthorized access. In NIST, encryption-at-rest is explained in [SC-28 Protection of Information at Rest](#). SC-28(1) aims to protect information at rest through cryptography and SC-28(2) aims to protect information via offline storage. Both of these controls are required for Solution providers who handle Amazon customer PII, as explained in the SP-API DPP. Best practice is to encrypt Amazon Information prior to uploading it to a storage destination and protect the storage itself with an approved encryption algorithm. This protects the data at two levels: during transmission from the source to the storage destination and throughout its lifespan at rest in the storage destination.

Solution providers using Amazon S3 for storage can use the native encryption capabilities of S3 to protect data at rest. S3 encrypts data at the object level as it writes to disk in AWS data centers, rather than encrypting entire buckets. Solution providers can choose to encrypt in two ways, using the server-side encryption (SSE) example just described, or client-side encryption (CSE).

Feeds API	
Feed Type Values	
Feeds JSON Schemas	
Feeds API Best Practices	
Feeds API FAQ	
Feeds API Rate Limits	
FINANCES	
Finances API	
Transfers API	
FULFILLMENT INBOUND API	
Fulfillment Inbound API	
Fulfillment Inbound API FAQ	
Migrating Fulfillment Inbound workflows	
Fulfillment Inbound API Rate Limits	
FULFILLMENT OUTBOUND API	
Fulfillment Outbound Dynamic Sandbox Guide	
Fulfillment Outbound API	

- > Solution providers can choose CSE when they want more control over encryption algorithms, key management, and implementation. With CSE, providers write code to encrypt and decrypt data before uploading to S3. CSE occurs when the object is encrypted prior to uploading to S3 (thus the keys are not managed by AWS). Solution providers can implement an Amazon S3 bucket policy that prevents uploading unencrypted objects. If a Solution provider does not need to manage the primary key then they can use server-side encryption with S3-managed encryption keys (SSE-S3) or server-side encryption with AWS KMS-managed keys (SSE-KMS). Amazon S3 protects data at rest using envelope encryption. Each object is encrypted with a unique key employing strong multi-factor encryption. As an additional safeguard, AWS encrypts the key with an AWS KMS key. Amazon S3 server-side encryption uses AES-256 to encrypt data. In Figure 2, the encryption solution protects the encryption key as well as the data. Solution providers storing data in other channels or cloud providers can utilize similar tools native to them (such as Azure Key Vault and GCP Default Encryption).



For more information, refer to: [How to prevent uploads of unencrypted objects to Amazon S3.](#)

Key Management Systems (KMS)

AWS Key Management Service (KMS) is a managed service that makes it easy to create and control the encryption keys used to encrypt data. KMS provides a centralized system for managing the complete key lifecycle, including key generation, storage, rotation, and revocation.

Solution providers can use KMS to:

- **Credential retrieval:** KMS enables secure retrieval of credentials needed to connect to databases, other systems, and APIs without hardcoding sensitive information.
- **Manage encryption keys centrally:** KMS provides a single point of control for creating, storing, and managing cryptographic keys used across your applications.
- **Implement key rotation:** KMS supports both automatic and manual key rotation to help meet compliance requirements and security best practices.
- **Control access with fine-grained permissions:** KMS integrates with AWS Identity and Access Management (IAM) to control who can use and manage your encryption keys.
- **Audit key usage:** KMS logs all key usage to AWS CloudTrail, providing a complete audit trail for compliance and security analysis.

AWS KMS generates symmetric keys for encryption, also called Data Keys. Solution providers must implement KMS with proper access controls, ensuring that cryptographic materials are only accessible to authorized processes and services. Keys must be protected throughout their lifecycle events: generation, exchange, storage, distribution, backup, recovery, use, replacement, and destruction.

Types of encryption

These are the common types of encryption:

- **Symmetric encryption.** Uses the same key for encryption and decryption. Therefore, both the sender and the receiver must have the same key.
- **Asymmetric encryption.** Requires different keys for sender and receiver (a public key and a private key). The public key is shared widely but the private key is protected and used only by its owner. The public key is used to encrypt the data and the private key is used to decrypt the data.
- **Envelope encryption.** Encrypts data with one key, then encrypts that key

Fulfillment Outbound API Rate Limits

MCF Best Practices

INVOICING

Invoices API >

Invoices API FAQ

LISTINGS

Listings Items API >

Listings Items API Rate Limits

Listings Restrictions API >

Listings Restrictions API Rate Limits

Manage Product Listings with the Selling Partner API

Building Listings Management Workflows Guide

Understanding Amazon listing status and seller-fulfilled inventory management

Listings Management Workflow Migration >

Manage Amazon Haul, Advanced Multiple-Offer, and Multiple-Fulfillment Use Cases

Listings APIs FAQ

Product Type Definitions API >

Product Type Definitions API Rate Limits

MERCHANT FULFILLMENT API

Merchant Fulfillment API >

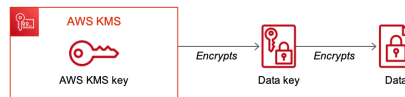
MESSAGING API

Messaging API >

In general, symmetric key algorithms are faster and produce smaller cipher texts than asymmetric algorithms. But asymmetric algorithms provide inherent separation of roles and easier key management. Envelope encryption lets a Solution provider combine the strengths of both symmetric and asymmetric algorithms. When encrypting Amazon information, Solution providers must also protect the encryption key. Envelope encryption helps Solution providers protect the encryption key itself.

Solution providers using SSE-KMS can encrypt data using one encryption key and encrypt the encryption key using another key. One key remains in plaintext so the client can decrypt the encryption key and use the decrypted encryption key to decrypt the data.

The top-level plaintext key is known as the AWS KMS key.



For more information, refer to [AWS Key Management Service](#).

Key size is important in defining the strength of an encryption key. As required by the Amazon DPP, keys must be at least 128 bits in length. Amazon recommends that Solution providers use keys larger than 128 bits for Amazon customer PII (256 bits preferred). The additional bits exponentially increase the complexity of a brute force attack against the encryption key. A symmetric 256-bit key requires 2128 times more computational power than a 128-bit key. Solution providers should rotate their cryptographic keys to further protect their systems. Although larger key sizes are harder to break, an exposed key renders key size irrelevant, as unauthorized entities can use it to access Amazon Information. Specific guidance on key rotation can be found in NIST 800-53 control [SC-12 Cryptographic Key Establishment and Management](#). The SC-12 control and its enhancements can help Solution providers establish, protect, and

rotate keys in a manner that is acceptable for their organization, while ensuring that the keys remain available when required.

NOTIFICATIONS API

Notifications API

Notification Type Values

Tutorial: Subscribe to the ORDER_CHANGE Notification

ORDERS API

Orders API

Tutorial: Retrieve and Pass a Purchase Order Number to a Carrier

Orders API Migration Guide

Orders API Rate Limits

PRODUCT FEES API

Product Fees API

PRODUCT PRICING API

Product Pricing API

Product Pricing API and Notifications FAQ

Amazon API Applications acceptable encryption standards

The following is an overview of commonly used and recommended encryption standards that Solution providers should use:

Advanced Encryption Standard (AES)

The Advanced Encryption Standard (AES), is a symmetric encryption algorithm that uses a block cipher. The block cipher encrypts a fixed text block to an equal length cipher-text block. AES operates on a fixed-length block of 128 bits in length, with a configurable symmetric key length of 128, 192, or 256 bits. Even with a 128-bit key, the task of cracking AES by checking each of 2¹²⁸ possible key values (a brute force attack) is so computationally intensive that the fastest supercomputer would require, on average, more than 100 trillion years to do it. If Solution providers choose to use AES keys, the key size must be AES 128-bit or larger (as required by the Amazon DPP). Amazon recommends using AES-256. Amazon also approves the use of AES-GCM/CBC/XTS.

Rivest-Shamir-Adleman Encryption Standard (RSA)

The Rivest-Shamir-Adleman encryption standard (RSA), is an asymmetric algorithm that uses public key cryptography to share data over an insecure network. RSA uses large integers (products of two large prime numbers) to establish key size. It generally uses 1024-bit or 2048-bit keys. A larger key size provides greater security but uses more computational power to encrypt and decrypt information. If Solution providers choose RSA as their encryption standard, they must ensure that their keys are at least 2048-bit, as required by the Amazon DPP.

Other encryption algorithms

While other encryption standards are available to Solution providers, the Amazon

Price Adjustment Automation Workflows Guide
Manage automated pricing rules with SP-API

REPLENISHMENT API
Replenishment API

REPORTS API
Reports API

Report Type Values

Reports JSON Schemas
Reports API FAQ

SALES API
Sales API

SELLER WALLET API
Seller Wallet API

Protect Amazon API Applications with Data Encryption

DPP requires that Solution providers use AES or RSA. Table 2 lists Amazon-approved encryption algorithms. If Solution providers are using other industry-standard encryption algorithms that are not listed in the following table, they should contact security@amazon.com for consultation.

Encryption algorithm	Amazon-approved type
AES	256-bit (preferred) 128-bit or larger keys
AES with GCM mode	96-bit cryptographically random initialization vector 128-bit tag length
AES with CBC mode	128-bit cryptographically random initialization vector PKCS7 padding
AES with XTS mode	Linux: dm-crypt, LUKS Amazon EBS encryption
RSA	2048-bit or larger keys RSA with OAEP

AWS Software Development Kit ([SDK](#)) offers a client-side encryption library that simplifies the implementation of encryption. By default, it implements a framework that keeps data secure using industry cryptography best practices for unique encryption keys and key protection. The SDK is developed on GitHub open source libraries so that you can analyze the code, submit issues, and find information specific to your implementation.

Standards not to use

These standards have been researched by the industry and proven insecure:

- DES
- RC4
- RSA-PKCSv1.5 with 1024-bit keys
- RSA-OAEP with 1024-bit keys
- Blowfish
- Twofish

Best practices for encryption

Solution providers should follow these best practices for robust protection of in-transit and at-rest data:

Seller Wallet API Rate Limits

SELLERS API

Sellers API

>

- Implement robust network security controls to help protect data in transit. For example, Solution providers can use firewalls and network access control lists to secure their networks against malware attacks or intrusions.

SERVICES API

Services API

>

- Implement proactive security measures that identify at-risk data and implement effective data protection for data in transit and at rest. Don't wait for a security incident to implement security measures.

SHIPMENT INVOICING API

Shipment Invoicing API

>

- Choose solutions with policies that enable user prompts and blocks, or that automate encryption of PII data in transit. For example, when users attach files containing PII to email messages, large or sensitive data file sets transferred to external file sharing sites, the application must display a warning or block the action.

SHIPPING API

Shipping API v2 Resources

↗

Shipping API v1 Reference

- Create policies for systematically categorizing and classifying all Amazon Information.

SOLICITATIONS API

Solicitations API

>

- Ensure appropriate data protection measures are applied while data remains at rest, regardless of where the data is stored.
- Implement triggers that alert when unauthorized parties access, use, or transfer data.

SUPPLY SOURCES API

Supply Sources API

>

- Carefully evaluate public, private, or hybrid cloud providers used for storing Amazon Information, based on the security measures that they offer to secure the data.

Multi-Location Inventory Integration Guide

Supply Sources API Rate Limits

- Use AWS KMS or equivalent key management systems to securely manage encryption keys, fetch credentials for running code, and maintain proper access controls throughout the key lifecycle.

TOKENS API

Tokens API >

Protect Amazon API Applications with Data Encryption

Data in transit and at rest have slightly different risk profiles, yet the inherent risk hinges primarily on the sensitivity and value of Amazon Information. Bad actors attempt to gain access to valuable data in transit or at rest. It is important to classify and categorize data and security protocols to effectively protect Amazon Information at every stage. Steps you can follow:

UPLOADS API

Uploads API >

VEHICLES API

Vehicles API >

Vehicles API Rate Limits

VENDOR DIRECT FULFILLMENT APIS

Vendor Direct Fulfillment Dynamic Sandbox Guide

Vendor Direct Fulfillment Workflow Guide

SP-API Bill-to-Party Addresses

VENDOR DIRECT FULFILLMENT TRANSACTION STATUS API

Vendor Direct Fulfillment Transaction Status API >

VENDOR DIRECT FULFILLMENT INVENTORY API

Vendor Direct Fulfillment Inventory API >

VENDOR DIRECT FULFILLMENT ORDERS API

Vendor Direct Fulfillment Orders API >

VENDOR DIRECT FULFILLMENT SHIPPING API

Vendor Direct Fulfillment Shipping API >

- Classify Amazon Information into sensitivity levels and encrypt the critical data. Always encrypt Amazon customer PII using Amazon-approved encryption mechanisms.
- Establish automated policies that prompt or automate encryption for critical data in transit.
- Encrypt and decrypt using a publicly available and peer-reviewed algorithm such as AES and a secret key.
- Solution providers using cloud infrastructures must evaluate cloud vendors based on the security measures they offer and the security compliance certifications they hold.
- Implement Key Management Systems that cover the complete key lifecycle, including secure key generation, storage, rotation, and revocation processes.

Further reading

For additional information, refer to:

- [Data Protection Policy](#)
- [Acceptable Use Policy](#)
- [Best Practices for Security, Identity & Compliance](#)
- [Protect Data in Transit](#)
- [Protect Data at Rest](#)
- [NIST 800-53 Rev.5: Security and Privacy Controls for Information Systems and Organizations](#)
- [AWS Key Management Service Documentation](#)

 Updated 18 days ago

←

Network
Protection
Guidance

→

Protect
Amazon API
Applications
with Data
Retention and
Recovery

Did this page help you?

👍

Yes

👎

No

VENDOR DIRECT FULFILLMENT PAYMENTS API

Vendor Direct Fulfillment Payments API >

Policies and Agreements

[Privacy notice](#) | [Conditions of use](#)

VENDOR RETAIL PROCUREMENT INVOICES API

Vendor Retail Procurement Invoices API >

VENDOR RETAIL PROCUREMENT ORDERS API

Vendor Retail Procurement Orders API >

VENDOR RETAIL PROCUREMENT SHIPMENTS API

Vendor Retail Procurement Shipments API >

VENDOR RETAIL PROCUREMENT TRANSACTION STATUS API

Vendor Retail Procurement Transaction Status API >

